

Data and Artificial Intelligence

Cyber Shujaa Program

Week 1 Assignment

Web Scraping and Data Handling in Python

Student Name: Salome Kungu

Student ID: CS-DA03-26054

Contents

Data and Artificial Intelligence	1
Cyber Shujaa Program.....	1
Week 1 Assignment Web Scraping and Data Handling in Python.....	1
Introduction	2
1. Set up and use Google Colab for Python development.....	2
2. Use the Requests library to fetch webpage content.....	2
3. Use BeautifulSoup to parse HTML and locate specific data points.....	2
4. Clean and organize data into a Pandas DataFrame.....	2
5. Export the final dataset as a .csv file.....	2
Tasks Completed	2
Conclusion.....	3

Introduction

The goal of this assignment was to automate data collection from the web using Python. As a newcomer to coding and Google Colab, this project provided a practical "learning by doing" experience. I successfully navigated the process of extracting structured data from [ScrapeThisSite](#) and transforming it into a usable format.

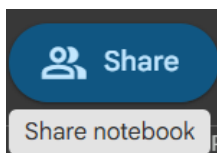
The objectives of the assignment were:

1. Set up and use Google Colab for Python development.
2. Use the Requests library to fetch webpage content.
3. Use BeautifulSoup to parse HTML and locate specific data points.
4. Clean and organize data into a Pandas DataFrame.
5. Export the final dataset as a .csv file.

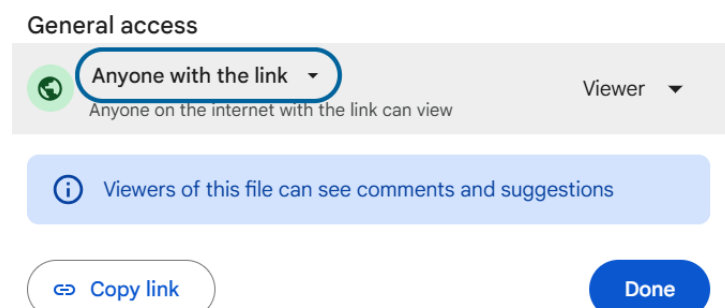
Tasks Completed

The technical process began by setting up a Python environment in **Google Colab**, where I imported the requests, BeautifulSoup, and pandas libraries. I initiated the data collection by sending an HTTP GET request to the ScrapeThisSite URL, successfully capturing the page's HTML content. Once the data was loaded, I used **BeautifulSoup** to parse the HTML and navigate the site's structure. I specifically targeted the table rows (<tr>) to isolate team statistics.

To ensure the data was clean, I wrote a loop to extract the text from each cell, using the .strip() method to remove unnecessary whitespace and formatting. These cleaned data points—including Team Names, Wins, and Losses—were then organized into a structured **Pandas DataFrame**. This allowed me to view the data in a clear, tabular format directly within the notebook. Finally, I exported this refined dataset into a .csv file, making the information ready for external analysis. Throughout the process, I maintained good coding practices by using descriptive variable names and adding comments to explain each step of the automation.



Ensure you allow 'Anyone with the link' to be a Viewer



Link to Code:

<https://colab.research.google.com/drive/1NUL49W4kDamaXu4SF4T02PSpilc52DFW?usp=sharing>

Project Deliverables

- **Link to Code:**
<https://colab.research.google.com/drive/1NUL49W4kDamaXu4SF4T02PSpilc52DFW?usp=sharing>
- **Documentation:** https://www.linkedin.com/posts/salome-k-51269993_datascience-python-webscraping-activity-7417869199101906946-rDP0?utm_source=share&utm_medium=member_desktop&rcm=ACoAABPbUJ4BvsL6X59c7WuiyOX33BLWxChY0PM

Conclusion

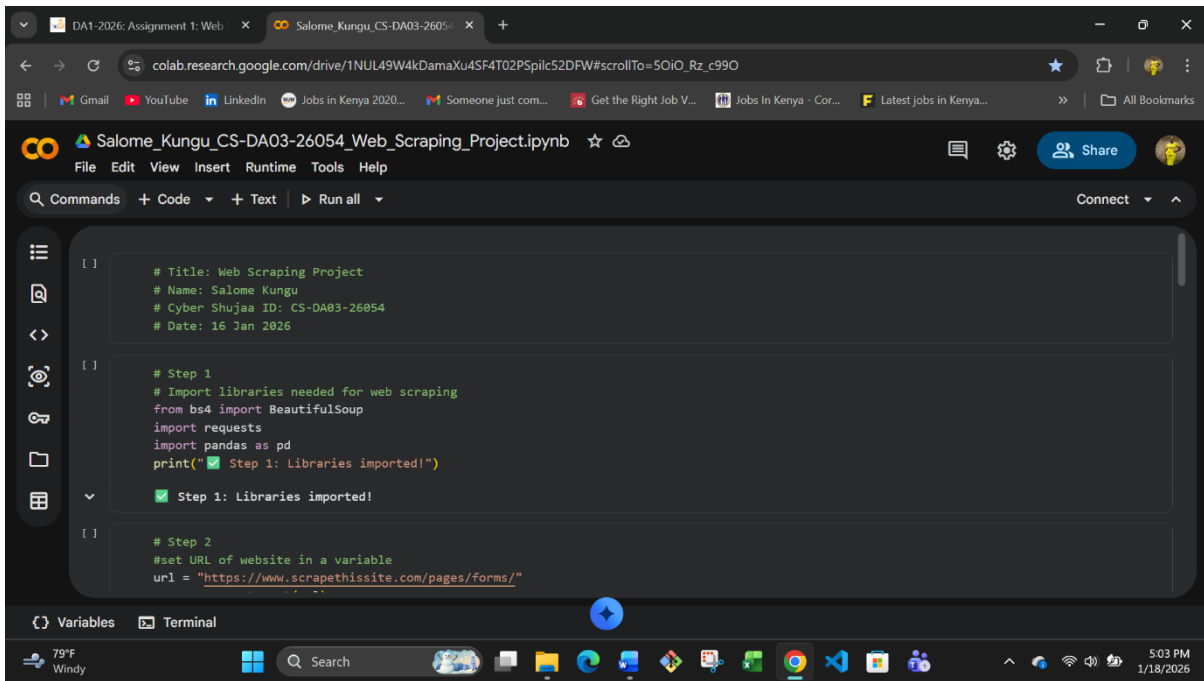
This week provided a solid grounding in the introductory concepts of data science. I have gained a better understanding of how to use Python to interact with web data—a skill I look forward to building upon as we move toward more advanced AI concepts. I have documented my learning journey on **LinkedIn** to begin building a professional portfolio that showcases my technical growth to future employers in the Data and AI industry.

Screenshots of the project

Step 1: Introduction of the project. Importation of libraries needed for web scrapping.

What: Introduced the project and imported the necessary Python libraries required for web scrapping and data handling.

Why: This prepares the environment by loading the tools needed to fetch web content, parse HTML, and process data efficiently.



The screenshot shows a Google Colab notebook titled "Salome_Kungu_CS-DA03-26054_Web_Scraping_Project.ipynb". The notebook is open in a web browser, and the code editor displays the following Python code:

```
[ ]  
  
# Title: Web Scraping Project  
# Name: Salome Kungu  
# Cyber Shujaa ID: CS-DA03-26054  
# Date: 16 Jan 2026  
  
[ ]  
  
# Step 1  
# Import libraries needed for web scraping  
from bs4 import BeautifulSoup  
import requests  
import pandas as pd  
print("✅ Step 1: Libraries imported!")  
  
✅ Step 1: Libraries imported!  
  
[ ]  
  
# Step 2  
#set URL of website in a variable  
url = "https://www.scrapethissite.com/pages/forms/"
```

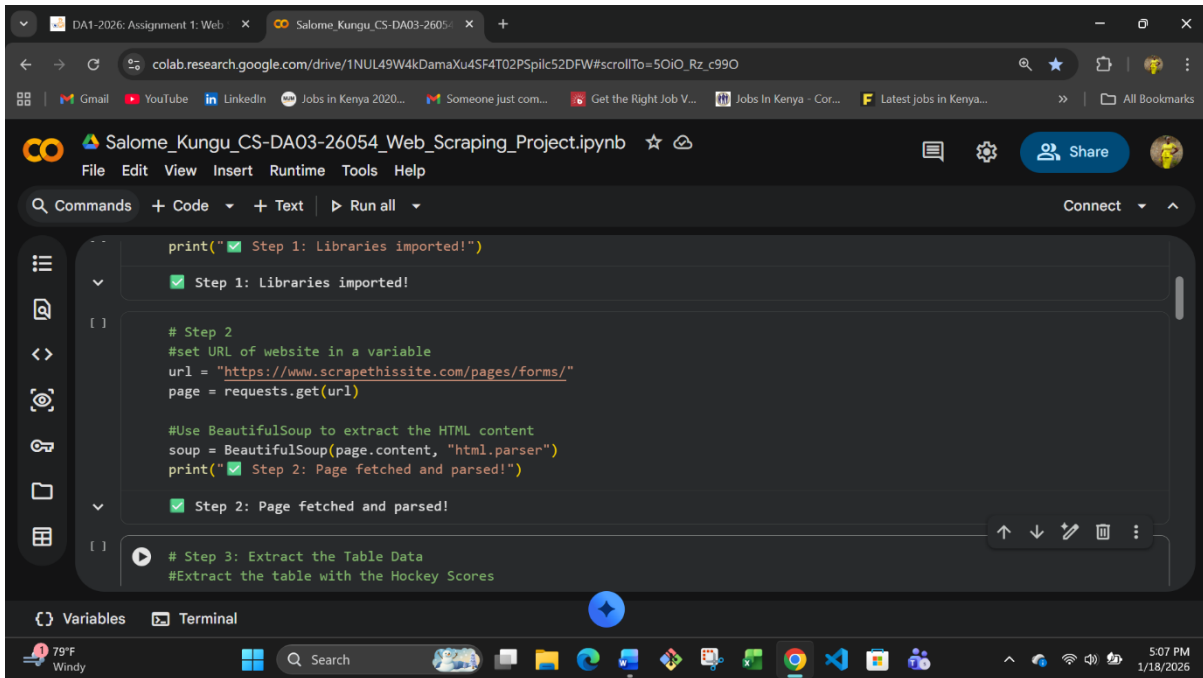
The notebook interface includes a menu bar with options like File, Edit, View, Insert, Runtime, Tools, and Help. A left sidebar contains icons for file management and viewing. The bottom of the screen shows a Windows taskbar with various application icons and system status information, including the temperature (79°F) and date (1/18/2026).

Step 2 : Set of URL website in a variable.

Use BeautifulSoup to extract to HTML content.

What: Stored the target website URL in a variable and used BeautifulSoup to extract and parse the HTML content.

Why: Defining the URL allows the script to access the webpage, while parsing the HTML makes it possible to locate and extract specific data elements.

A screenshot of a Google Colab notebook titled "Salome_Kungu_CS-DA03-26054_Web_Scraping_Project.ipynb". The notebook is open in a web browser, showing the URL "colab.research.google.com/drive/1NUL49W4kDamaXu4SF4T02P5pIc52DFW#scrollTo=50iO_Rz_c99O". The notebook interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help), a toolbar (Commands, Code, Text, Run all), and a sidebar with icons for file management and search. The code editor shows a Python script with three steps: Step 1 (Libraries imported), Step 2 (URL set and HTML content extracted), and Step 3 (Table data extracted). The script uses the 'requests' library to fetch the page and 'BeautifulSoup' to parse the HTML. The output of the script is visible in the console, showing the execution of each step.

```
print("✅ Step 1: Libraries imported!")

✅ Step 1: Libraries imported!

[ ]

# Step 2
#set URL of website in a variable
url = "https://www.scrapethissite.com/pages/forms/"
page = requests.get(url)

#Use BeautifulSoup to extract the HTML content
soup = BeautifulSoup(page.content, "html.parser")
print("✅ Step 2: Page fetched and parsed!")

✅ Step 2: Page fetched and parsed!

[ ]

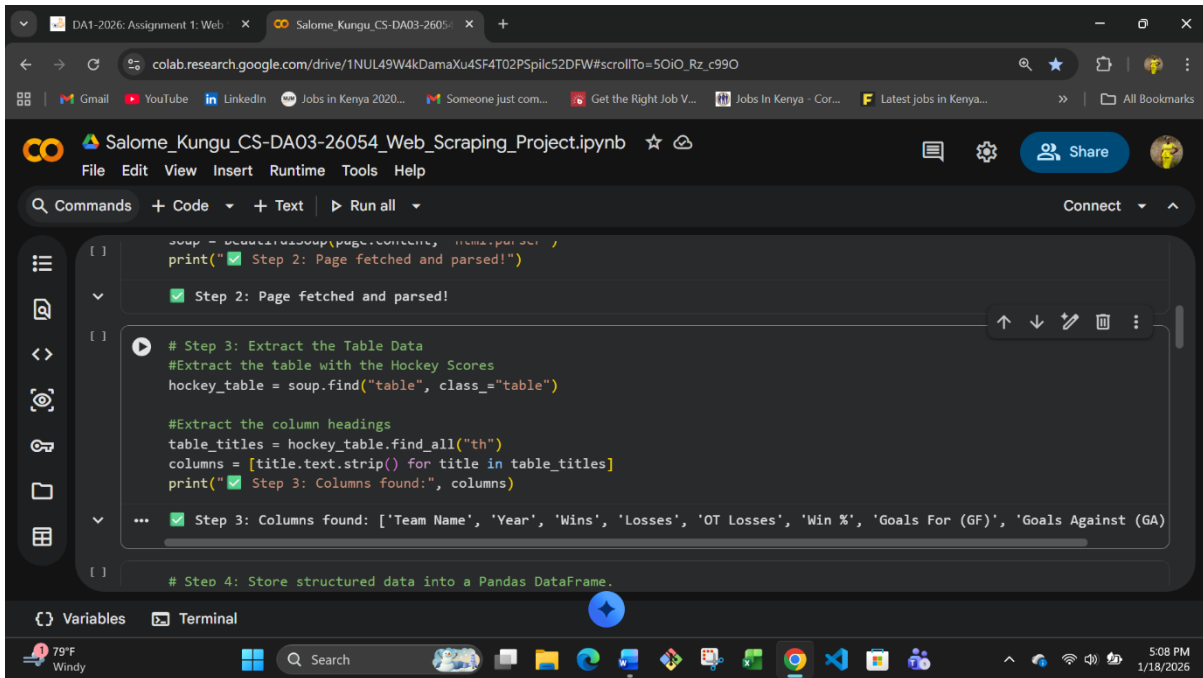
# Step 3: Extract the Table Data
#Extract the table with the Hockey Scores
```

Step 3: Extract the table data.

Extracting the table with the hockey scores and extract column headings.

What: Identified and extracted the table containing the hockey scores, including the column headings.

Why: Extracting the table and headers structures the raw webpage data, making it easier to organize and analyze.



```
[ ] soup = BeautifulSoup(page_content, 'html.parser')
print("✅ Step 2: Page fetched and parsed!")

[ ] # Step 2: Page fetched and parsed!

[ ] # Step 3: Extract the Table Data
#Extract the table with the Hockey Scores
hockey_table = soup.find("table", class_="table")

#Extract the column headings
table_titles = hockey_table.find_all("th")
columns = [title.text.strip() for title in table_titles]
print("✅ Step 3: Columns found:", columns)

... [ ] # Step 3: Columns found: ['Team Name', 'Year', 'Wins', 'Losses', 'OT Losses', 'Win %', 'Goals For (GF)', 'Goals Against (GA)']

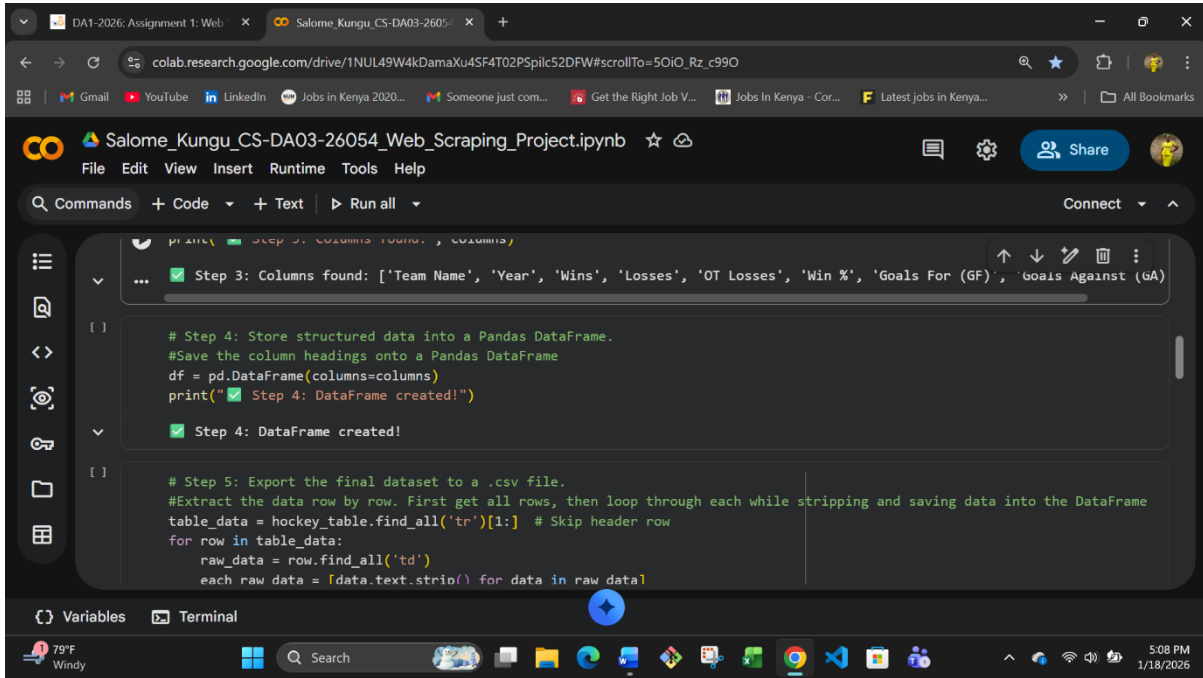
[ ] # Step 4: Store structured data into a Pandas DataFrame.
```

Step 4: Store structured data into a pandas dataframe.

Saving the column headings into a pandas dataframe

What: Saved the extracted column headings and structured data into a pandas DataFrame.

Why: A DataFrame provides a structured and tabular format that simplifies data manipulation, inspection, and export.



```
print('Step 3: Columns found:', columns)

...
[ ]
# Step 4: Store structured data into a Pandas DataFrame.
# Save the column headings onto a Pandas DataFrame
df = pd.DataFrame(columns=columns)
print("Step 4: DataFrame created!")

[ ]
Step 4: DataFrame created!

[ ]
# Step 5: Export the final dataset to a .csv file.
# Extract the data row by row. First get all rows, then loop through each while stripping and saving data into the DataFrame
table_data = hockey_table.find_all('tr')[1:] # Skip header row
for row in table_data:
    raw_data = row.find_all('td')
    each raw data = 'data.text.strip()' for data in raw data
```

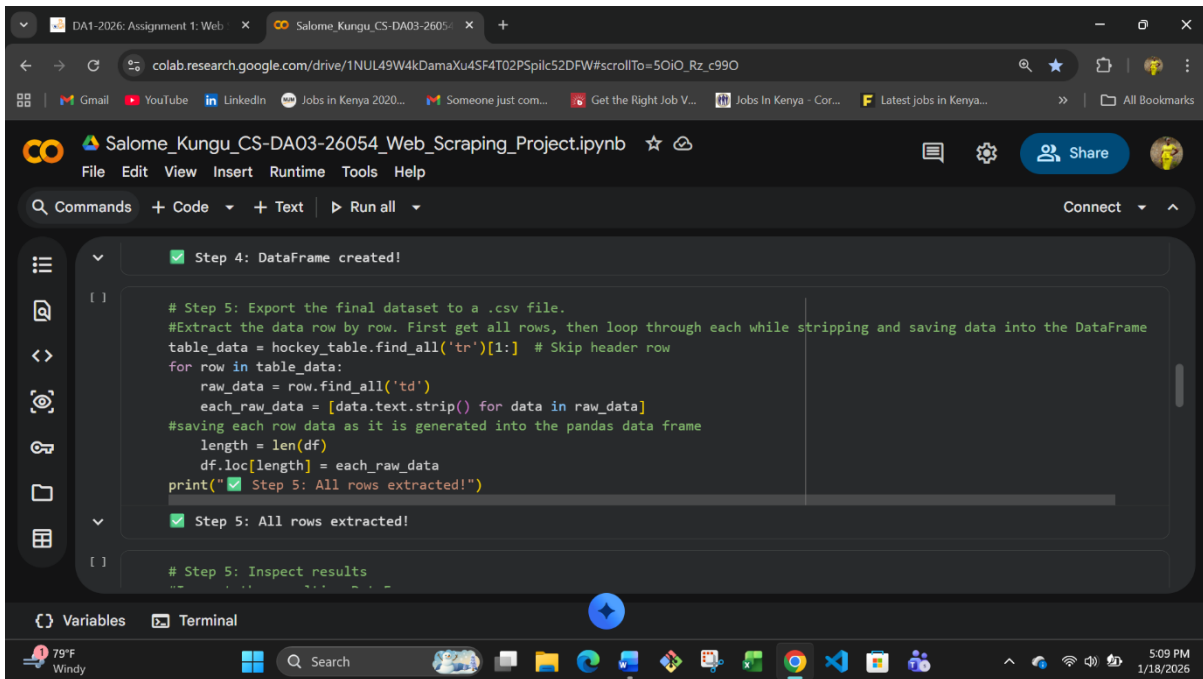
Step 5: Export of the final dataset to a .csv file.

Extract the data row by row, then loop through each while stripping and saving data into the dataframe.

Save each row data as it is generated into the pandas dataframe.

What: Extracted table data row by row, looping through each entry while stripping unnecessary characters and saving the cleaned data into the DataFrame.

Why: This ensures that each data point is accurately captured, cleaned, and stored in a consistent format.



The screenshot shows a Google Colab notebook titled "Salome_Kungu_CS-DA03-26054_Web_Scraping_Project.ipynb". The notebook is open to a code cell containing the following Python code:

```
[ ]  
  
# Step 5: Export the final dataset to a .csv file.  
#Extract the data row by row. First get all rows, then loop through each while stripping and saving data into the DataFrame  
table_data = hockey_table.find_all('tr')[1:] # Skip header row  
for row in table_data:  
    raw_data = row.find_all('td')  
    each_raw_data = [data.text.strip() for data in raw_data]  
    #saving each row data as it is generated into the pandas data frame  
    length = len(df)  
    df.loc[length] = each_raw_data  
    print("✅ Step 5: All rows extracted!")  
  
[ ]  
  
# Step 5: Inspect results  
--
```

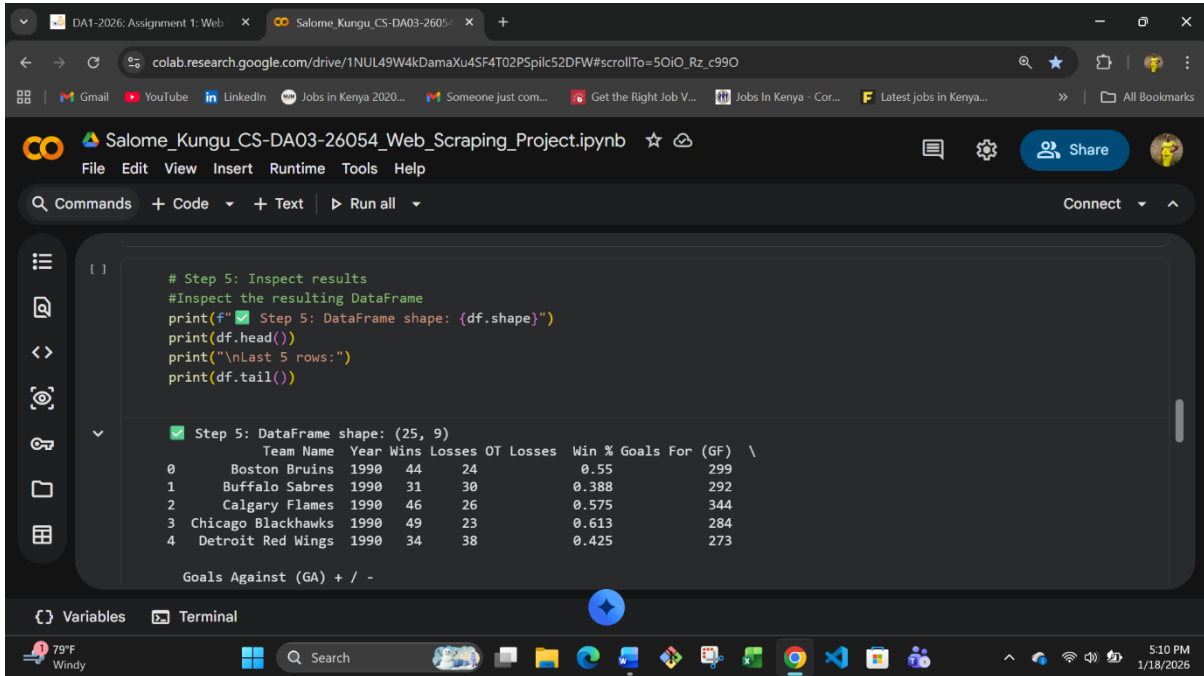
The code cell is followed by a status bar indicating "Step 5: All rows extracted!". The notebook interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help), a toolbar (Commands, Code, Text, Run all), and a bottom panel (Variables, Terminal). The bottom status bar shows the system clock as 5:09 PM on 1/18/2026.

Step 5: Inspect the resulting dataframe

Inspect results output of the dataframe

What: Inspected the output of the DataFrame to verify the extracted data.

Why: Reviewing the DataFrame helps confirm data accuracy and completeness before final export.

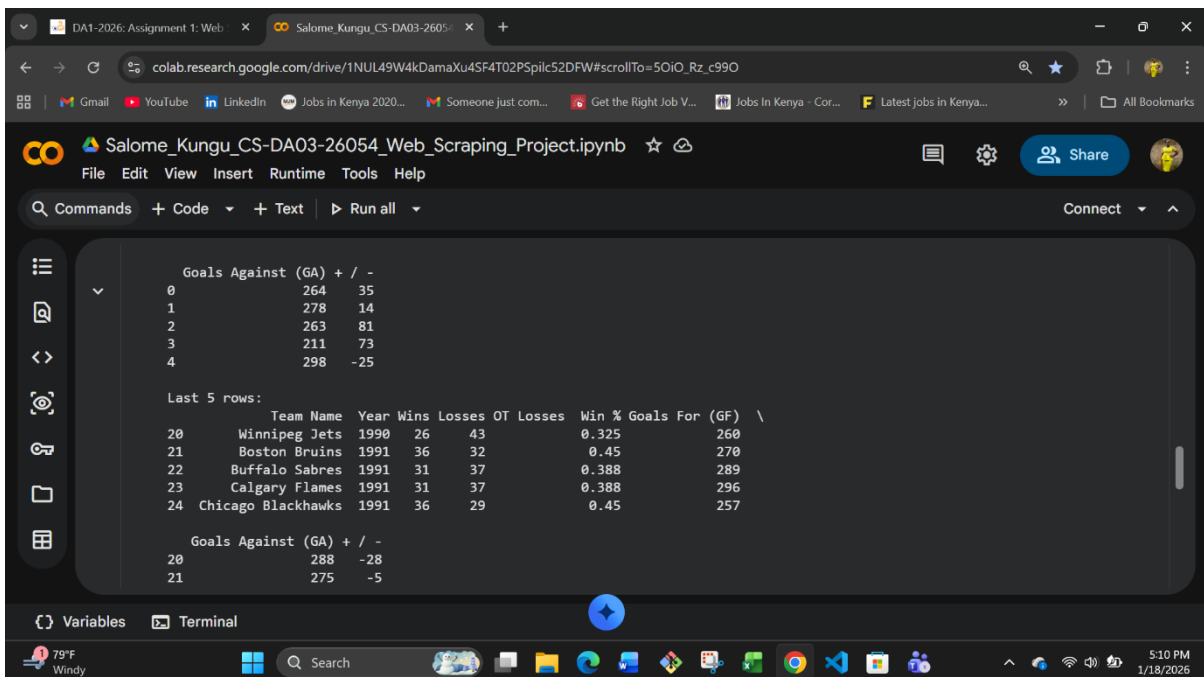


```
# Step 5: Inspect results
#Inspect the resulting DataFrame
print(f'✅ Step 5: DataFrame shape: {df.shape}')
print(df.head())
print("\nLast 5 rows:")
print(df.tail())
```

✅ Step 5: DataFrame shape: (25, 9)

	Team Name	Year	Wins	Losses	OT Losses	Win %	Goals For (GF) \
0	Boston Bruins	1990	44	24		0.55	299
1	Buffalo Sabres	1990	31	30		0.388	292
2	Calgary Flames	1990	46	26		0.575	344
3	Chicago Blackhawks	1990	49	23		0.613	284
4	Detroit Red Wings	1990	34	38		0.425	273

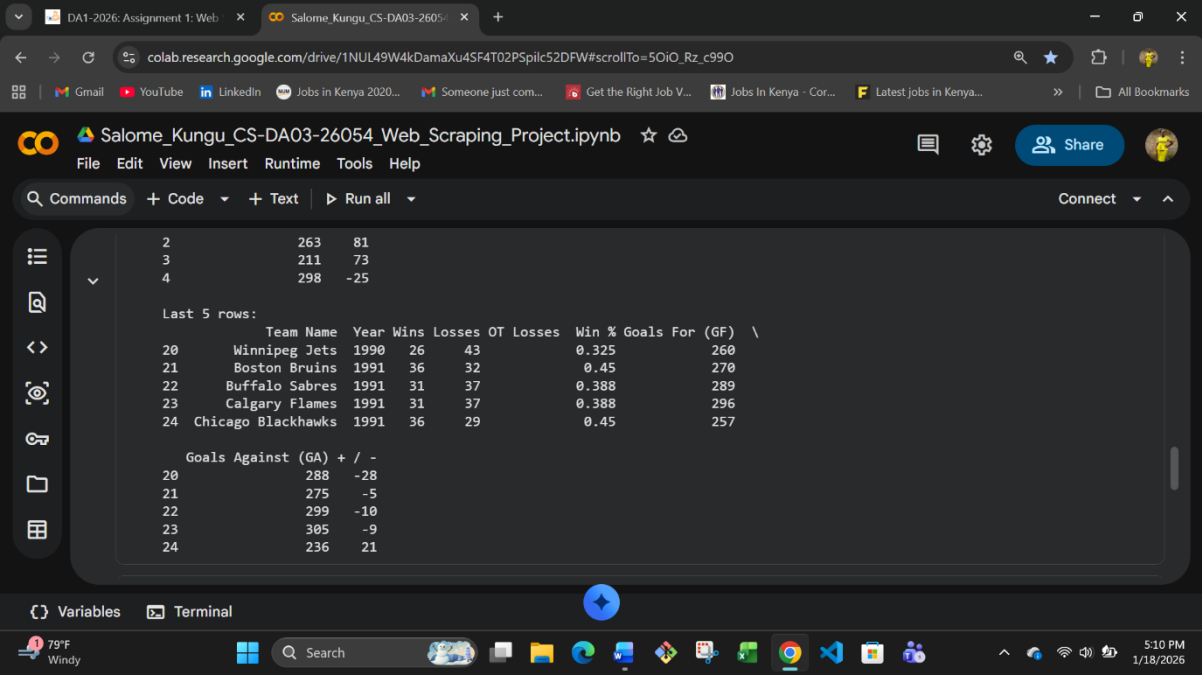
Goals Against (GA) + / -



```
Goals Against (GA) + / -
0      264    35
1      278    14
2      263    81
3      211    73
4      298   -25

Last 5 rows:
      Team Name Year Wins Losses OT Losses Win % Goals For (GF) \
20  Winnipeg Jets 1990  26   43      0.325      260
21  Boston Bruins 1991  36   32      0.45       270
22  Buffalo Sabres 1991  31   37      0.388      289
23  Calgary Flames 1991  31   37      0.388      296
24  Chicago Blackhawks 1991  36   29      0.45      257

Goals Against (GA) + / -
20      288   -28
21      275    -5
```



colab.research.google.com/drive/1NUL49W4kDamaXu4SF4T02P5pilc52DFW#scrollTo=50iO_Rz_c99O

Salome_Kungu_CS-DA03-26054_Web_Scraping_Project.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Connect

```

2      263    81
3      211    73
4      298   -25

Last 5 rows:
   Team Name  Year  Wins  Losses  OT Losses  Win %  Goals For (GF) \
20  Winnipeg Jets  1990   26    43         0.325      260
21  Boston Bruins  1991   36    32         0.45       270
22  Buffalo Sabres  1991   31    37         0.388      289
23  Calgary Flames  1991   31    37         0.388      296
24  Chicago Blackhawks  1991   36    29         0.45      257

Goals Against (GA) + / -
20      288   -28
21      275    -5
22      299   -10
23      305    -9
24      236    21

```

Variables Terminal

79°F Windy

Search

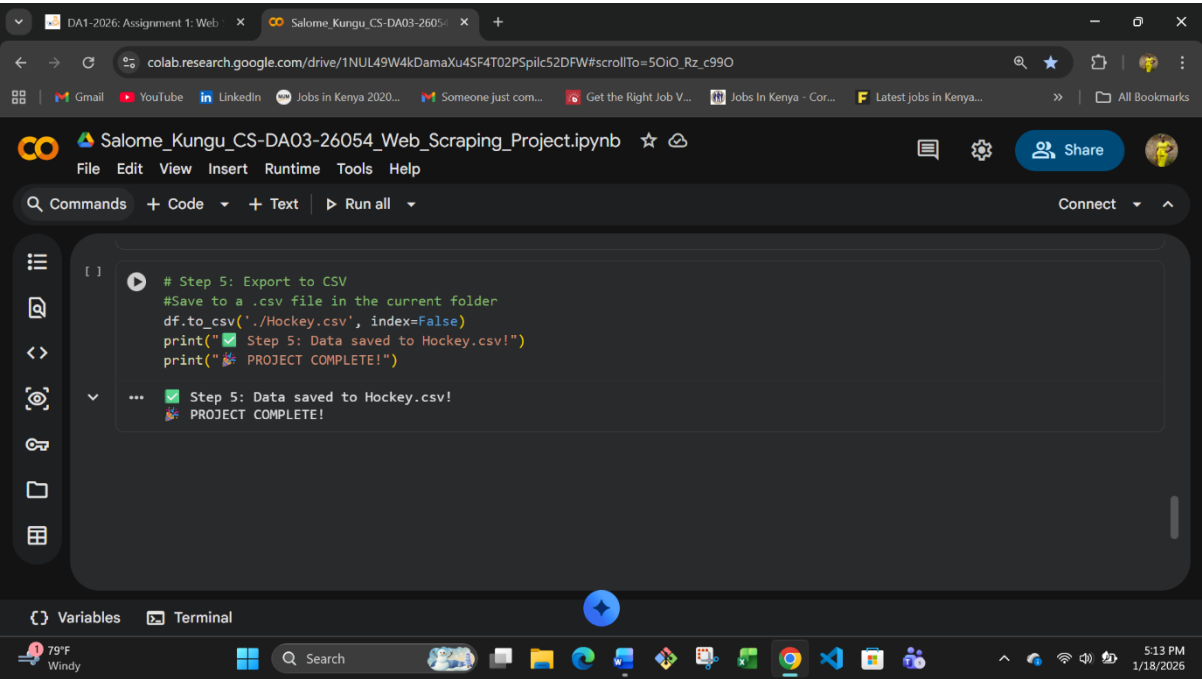
5:10 PM 1/18/2026

Step 5: Export to CSV

Save to a .csv file.

What: Exported the final DataFrame to a .csv file.

Why: Saving the data as a CSV file allows for easy sharing, storage, and further analysis using other tools or software.



colab.research.google.com/drive/1NUL49W4kDamaXu4SF4T02P5pilc52DFW#scrollTo=50iO_Rz_c99O

Salome_Kungu_CS-DA03-26054_Web_Scraping_Project.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Connect

```

[ ] # Step 5: Export to CSV
    # Save to a .csv file in the current folder
    df.to_csv('./Hockey.csv', index=False)
    print("✅ Step 5: Data saved to Hockey.csv!")
    print("🎉 PROJECT COMPLETE!")

```

... ✅ Step 5: Data saved to Hockey.csv!
🎉 PROJECT COMPLETE!

Variables Terminal

79°F Windy

Search

5:13 PM 1/18/2026